# Tuning Small Embedders for Code Search with LLM-Written Questions: How Much Is Repository-Specific?

Vinicius Pedrini, Independent Researcher. viniciusvmp@poli.ufrj.br
Preliminary results, 12 November 2026. Code, questions and result files: https://github.com/vinimpv/tuna
Experiments were run and text was drafted with AI coding assistants. The author is responsible for all content.

## Summary

We tune three small embedding models for code search inside a repository, using only questions that a locally served LLM writes about the repository's code. We test on questions that a different LLM wrote about files left out of tuning, on two repositories that no model was tuned on, and on real GitHub issues. The main findings so far: (1) Tuning raises MRR@10 for every model on every repository, by 0.19 to 0.30 pooled over the three. (2) Most of that gain is not repository-specific. A model tuned on a different repository gets 79% to 89% of it, and gains 0.16 to 0.27 on two repositories outside Django. Tuning on the target adds 0.02 to 0.05. (3) A 350M-parameter general-text embedder starts below BM25 and ends near stock embedders of 0.6B and 1.5B parameters. (4) On real issues only the smallest model gains. (5) A coding agent given search hits from the tuned index, instead of the stock one, found the right file no more often.

## Setup

**Repositories.** Saleor, Zulip and PostHog, each pinned to one commit. We index Python functions and classes (“symbols”): 11,678 in Saleor, 6,836 in Zulip and 29,679 in PostHog. All three are web backends built on Django.

**Training questions.** Qwen3.6-35B-A3B, served locally, writes 1,333 to 1,500 questions per repository. Each question has one gold symbol, and the prompt bans code identifiers in the question. About 10% of the questions form the validation split. The training questions touch about 5% to 18% of a repository's symbols.

**Held-out files.** 162 to 178 files per repository get no training question and supply no hard negative. Every question in the main test set is about a symbol in one of these files.

**Test sets.** (a) 400 questions per repository written by Claude Opus 5.5 about symbols in the held-out files, 1,200 in total. (b) 321 real GitHub issues, each mapped to the code its fixing commit changed. (c) 400 questions by the same writer on each of Netflix Dispatch and a subset of Home Assistant, two Python repositories outside Django that no model is tuned on.

**Models.** LFM2.5-Embedding-350M, Jina code embeddings 0.5B and Qwen3-Embedding-0.6B, each stock and tuned. BM25, Jina code embeddings 1.5B and LFM2.5-ColBERT-350M are stock baselines.

**Tuning.** One LoRA recipe for all three models. We picked it on Saleor's validation split alone. Tuning all weights instead is never reliably better (LoRA minus full, pooled: −.017 to +.032), and an adapter takes 11 to 19 MiB on disk against 0.7 to 1.1 GiB. Each model mines hard negatives with its own stock vectors, and stock Qwen3-Embedding-0.6B removes likely false negatives for all three, by the rule of NV-Retriever (arXiv:2407.15831). On one RTX 5090, writing the questions takes 29 to 61 minutes per repository, and one tuning run takes 6 to 18 minutes for LFM, 12 to 20 for Jina and 16 to 56 for Qwen3. From a fresh repository to a tuned LFM takes 36 to 89 minutes.

**Statistics.** Paired bootstrap, 10,000 samples, resampled by gold file within each repository, 95% intervals. For closeness we use a margin of 0.05 MRR@10, fixed before any interval was computed but after the point estimates were seen.

## Results

| Model | Saleor | Zulip | PostHog | Mean | Issues |
|---|---|---|---|---|---|
| BM25 | .265 | .369 | .439 | .357 | .279 |
| LFM2.5-Embedding-350M, stock | .247 | .301 | .328 | .292 | .207 |
| LFM2.5-Embedding-350M, tuned | .627 | .620 | .538 | .595 | .322 |
| Jina code 0.5B, stock | .510 | .546 | .523 | .527 | .349 |
| Jina code 0.5B, tuned | .735 | .731 | .691 | .719 | .367 |
| Qwen3-Embedding-0.6B, stock | .595 | .619 | .542 | .585 | .350 |
| Qwen3-Embedding-0.6B, tuned | .784 | .798 | .735 | .772 | .356 |
| Jina code 1.5B, stock | .617 | .658 | .570 | .615 | .349 |
| LFM2.5-ColBERT-350M, stock | .390 | .494 | .541 | .475 | .304 |

Table 1. MRR@10 of the gold symbol. Saleor, Zulip, PostHog and Mean: Claude-written questions on held-out files, 400 per repository. Issues: 321 real issues, pooled. LFM2.5-ColBERT reads only the first 32 tokens of a query, so its issue score comes from truncated issue text.

1. **Tuning helps every model on every repository.** Pooled over the 1,200 questions, tuned minus stock is +.303 [+.277, +.329] for LFM, +.192 [+.169, +.215] for Jina and +.187 [+.166, +.209] for Qwen3. The smallest gain in any single repository is +.167.
2. **Most of the gain is not repository-specific.** For each target repository we also score the adapters tuned on the other two. They reach 84%, 89% and 79% of the pooled gain (LFM, Jina, Qwen3), and tuning on the target itself adds +.049 [+.032, +.066], +.021 [+.006, +.036] and +.040 [+.025, +.054]. The gain also carries outside Django. On Netflix Dispatch and Home Assistant, the single-repository adapters gain +.16 to +.27 on average and one adapter trained on all three repositories gains +.18 to +.31, every interval above zero. On the three repositories' own written questions, that one adapter differs from the three separate ones by −.005 to +.039. On issues, Jina's is .03 worse (−.032 [−.054, −.010]).
3. **The smallest model ends near larger stock models.** Stock LFM2.5-Embedding-350M is below BM25, by −.065 [−.090, −.041]. Tuned, it lies within our margin of stock Qwen3-Embedding-0.6B (+.010 [−.014, +.034]) and of stock Jina code 1.5B (−.020 [−.046, +.007]), intervals included. On Saleor, the mean over three training seeds gives +.006 [−.036, +.048] and −.016 [−.058, +.026], and one of the six per-seed intervals stays within the margin. It is not the best model: tuned Qwen3-Embedding-0.6B leads in every repository, about .18 above tuned LFM. LFM answers a query in 6 ms against 13 ms for Qwen3-Embedding-0.6B, on the same GPU.
4. **On real issues only the smallest model gains.** LFM gains +.115 [+.078, +.152]. Jina (+.019 [−.008, +.045]) and Qwen3 (+.006 [−.029, +.044]) do not. Tuned LFM is not reliably above BM25 there (+.042 [−.003, +.088]), and no repository-specific part shows reliably. Three further attempts did not move the issue score reliably: bug-style training questions for LFM (+.008 [−.018, +.035]), a 4B stock model in place of the 0.6B one (+.001 [−.054, +.057], Saleor only), and more training questions for LFM (+.02 from 150 to 1,349 questions, Saleor only).
5. **In a coding agent, the better index did not change the outcome.** Claude Code localised all 321 issues alone, and with a short card of search hits placed before the task: from the stock LFM index, the tuned one, BM25, or BM25 fused with tuned Qwen3, always from the same queries. The tuned index put the gold symbol on the card more often than the stock one (+.069 [+.019, +.118]). The agent's gold-file rate did not follow: −.006 [−.032, +.020], the outcome we registered before the experiment. Alone the agent found the gold file in 64.5% of issues, and no card changed that reliably. Cards saved about half a call per task and no money.
6. **Tuning beats two alternatives and needs few questions (Saleor only).** Indexing one written question next to each of 11,151 symbols, with no training (doc2query), lifts stock LFM by +.246 [+.180, +.309] but leaves every stock model .13 to .17 below its tuned version, which used 1,500 questions. Stock Qwen3-Embedding-4B is below tuned Jina and tuned Qwen3-Embedding-0.6B, by .07 and .12. With 150 training questions LFM already has two thirds of its gain.

## Limits and work in progress

- One language. All five repositories are Python, and every model was tuned on Django backends.
- One kind of written question. Training and test questions share one prompt, and an LLM chose the symbol behind each. A model can gain by learning which symbols writers choose. In our test for such a preference, tuning raised it for Jina in all three repositories, for Qwen3 in one and for LFM in none. Where it rose, part of the gain on written questions may come from it. The writer matters too: about the same gold symbol, every model, stock or tuned, scores .06 to .15 lower on Claude's question than on Qwen's. Our headline trains on Qwen's questions and tests on Claude's.
- One training run per row, and our intervals do not cover the training seed. On Saleor, repeated runs (three seeds, two GPU models) moved the tuned score by up to .05 for LFM, .02 for Jina and .03 for Qwen3. Every gain from tuning held on each run, and Saleor's own adapter stayed ahead of the other two repositories' adapters on each. For LFM and Jina the reported run is the highest of them.
- No person has checked the test questions. A judge model from the same family as their writer (Claude Sonnet 5.5, judging questions written by Claude Opus 5.5) read 30 sampled questions from each of the five Claude-written test sets. It accepted the gold symbol as the answer for 149 of the 150, and it rejected all 15 wrong pairs mixed in as a control.
- The issue set is small and was visible during development. A fresh set, from fixes merged after the pinned commits, will be collected once and used once.
- The repositories are public, and are probably in the pretraining data of every model compared.

## Relation to prior work

Training retrieval models on LLM-written queries follows InPars (arXiv:2202.05144) and Promptagator (arXiv:2209.11755), and adapting a dense retriever to one corpus with generated queries follows GPL (arXiv:2112.07577). Indexing generated questions follows doc2query (arXiv:1904.08375). LoRACode (arXiv:2503.05315) tunes code embedders with LoRA, and SYNTRA (arXiv:2610.14730) scales synthetic retrieval training data with an agent loop. SweRank (arXiv:2505.07849) trains retrievers for issue localisation on real issues paired with their fixes. We do not propose a new method. We measure the known recipe inside single repositories: how much of the gain is specific to the repository, what a 350M model reaches, what it costs on one consumer GPU, and what changes on real issues and in a coding agent.